

Assignment: Spring Boot Application for Library Tracking

Student Profile: MD Kaif Molla

Institutional Roll: 2024eb02411

Source Code Repository: <https://github.com/WhySeriousKaif/library-management-assignment>

Part I: Database Schema and Architecture

In order to meet the project requirements, I selected a publishing domain, specifically managing the linkage between a `Book` and its `Author`.

- **Structural Relationship:** I established a one-to-many cardinality. Naturally, a single author writes many books, while each book in this system is mapped to precisely one primary author.
- **JPA Annotations utilized:**
 - The author side uses `@OneToMany(mappedBy = "author")` to keep track of the collection of books.
 - The book side uses `@ManyToOne` along with a `@JoinColumn` to enforce the foreign key constraint.

Here is a snippet showing how I mapped these entities in the code:

```
// Inside the Author entity
@OneToMany(mappedBy = "author", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
private List<Book> books = new ArrayList<>();

// Inside the Book entity
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "author_id", nullable = false)
private Author author;
```

Part II: Execution of Core Features

1. Seeding the H2 Database

Instead of relying on manual data entry for testing, I configured a `CommandLineRunner` that acts as an initialization script. Upon booting the server, it automatically injects a predefined list of 10 authors and 10 associated books directly into the tables.

```
@Bean
public CommandLineRunner initializeDatabase(AuthorRepository authorRepo,
BookRepository bookRepo) {
    return (args) -> {
        if (authorRepo.count() == 0) {
            // Logic to persist dummy authors and books goes here
        }
    };
}
```

2. The "Create" Workflow

To allow users to register new literature, I designed a JSP view (`create.jsp`). When the user submits this HTML form, it triggers a POST mapping in the controller. The controller delegates the persistence to the service layer. If constraints (like a duplicate ISBN) are violated, the exception is caught, and the user is redirected back to the form with a descriptive error alert.

```
@PostMapping("/create")
public String handleBookCreation(@ModelAttribute Book newBook, Model uiModel) {
    try {
        libraryService.saveBook(newBook);
        return "redirect:/";
    } catch (Exception ex) {
        uiModel.addAttribute("error", "Failed to add book: " + ex.getMessage());
        uiModel.addAttribute("authors", libraryService.getAllAuthors());
        return "create";
    }
}
```

3. The "Read" Workflow

Displaying the data efficiently requires fetching both the books and their respective authors simultaneously. To prevent the notorious N+1 query problem, I bypassed standard JPA methods and wrote a bespoke HQL query employing an inner join fetch mechanism.

```
// Inside BookRepository.java
@Query("SELECT b FROM Book b JOIN FETCH b.author")
List<Book> retrieveAllBooksWithAuthorDetails();
```

4. The "Update" Workflow

When a user decides to modify an existing entry, they are navigated to `update.jsp` where the inputs are pre-populated with the target book's current metadata. Upon saving, the changes overwrite the old values in the database.

```
@PostMapping("/update")
public String handleBookUpdate(@ModelAttribute Book updatedBook, Model uiModel) {
    try {
        libraryService.saveBook(updatedBook);
        return "redirect:/";
    } catch (Exception ex) {
        return "update";
    }
}
```

Part III: Quality Assurance and Testing

To ensure robust business logic, I integrated **JUnit 5** and **Mockito** for comprehensive testing:

- **Repository Testing:** The custom `JOIN FETCH` query was validated using the `@DataJpaTest` slice.
- **Service Testing:** `LibraryServiceTest` verifies behavior in isolation by mocking the underlying data repositories.

Furthermore, I conducted end-to-end manual checks via the browser interface to validate form submissions and UI behavior.

Part IV: Visual Evidence

Screenshot 1: The Main Dashboard (Viewing Data)

Library Management System

List of Books

Add New Book

ID	Title	ISBN	Author Name	Author Email	Actions
1	1728	978-0451524935i	Stephen King	king@example.com	Edit
2	Animal Farm	978-0451526342	George Orwell	orwell@example.com	Edit
3	Harry Potter and the Sorcerer's Stone	978-0590353427	J.K. Rowling	rowling@example.com	Edit
4	Harry Potter and the Chamber of Secrets	978-0439064873	J.K. Rowling	rowling@example.com	Edit
5	The Hobbit	978-0547928227	J.R.R. Tolkien	tolkien@example.com	Edit
6	The Fellowship of the Ring	978-0544003415	J.R.R. Tolkien	tolkien@example.com	Edit
7	Murder on the Orient Express	978-0062073495	Agatha Christie	christie@example.com	Edit
8	And Then There Were None	978-0062073488	Agatha Christie	christie@example.com	Edit
9	The Shining	978-0307743657	Stephen King	king@example.com	Edit
10	It	978-1501142970	Stephen King	king@example.com	Edit

Screenshot 2: The Registration Interface (Adding Data)

Add New Book

Book Title

ISBN

Select Author

-- Select Author --

Save BookCancel

Screenshot 3: The Modification Interface (Editing Data)

Edit Book Details

Book Title

Animal Farm

ISBN

978-0451526342

Select Author

George Orwell

Update BookCancel

Part V: Technical Roadblocks & Resolutions

1. **JSP Integration Complexity:** While Spring Boot naturally integrates with Thymeleaf, routing to traditional JSPs required a deeper dive into application properties (setting up prefixes and suffixes) and adding the `tomcat-embed-jasper` dependency to compile the templates at runtime.
2. **Query Optimization:** Naively calling `findAll()` on the book repository resulted in excessive independent database calls to fetch the associated authors. I mitigated this bottleneck by implementing an explicit `JOIN FETCH` query, successfully reducing the operation to a single SQL statement.
3. **Graceful Error Handling:** Database integrity errors (such as attempting to save a non-unique ISBN) initially crashed the application with a 500 status. I resolved this by wrapping the service calls in a `try-catch` block within the controller, allowing me to pass the exception message dynamically back to the user interface.